



Tries $\equiv$ Trees

- Viraj Chandra

Problem Statement



- Given a lot of strings $a_1, a_2, a_3 \dots a_n$, do the following for a new string X
 - ✓ ○ **Part 1:** Find if X exists in the set of strings
 - ✓ ○ **Part 2:** Find number of strings with prefix equal to X

Example:

- ✓ ● Google Search
- ✓ ● TLE's Leaderboard (Best Example)

$$a_1 = abcd \checkmark$$

$$a_2 = abd \checkmark$$

$$a_3 = ac \checkmark$$

$$a_4 = bac$$

$$a_5 = cda$$

$$X = pq\sigma \text{ (F)}$$

$$X = ac \text{ (T)}$$

$$X = a \text{ (3)}$$

$N \text{ strings} \rightarrow L \rightarrow \underline{1 \leq L \leq 10^3}$

$\downarrow$

$1 \leq N \leq 10^3$

$Q \rightarrow \underline{1 \leq Q \leq 10^3}$
 $1 \leq x_i \leq 10^3$

$\{ 2, 3, 5, 9, \dots, 18 \}$ set
 $\nearrow O(\log N)$ $\downarrow$

$\{ abc, abd, \textcircled{ac}, bd, \dots, de \}$
 $x = \uparrow$

$\nearrow$
TC?? $\rightarrow O(\log N \cdot |x|)$

Part-1 (Bonte)

Hashing? $\rightarrow O(L)$

{ abc, abd, ac, bd, bdc, dc }
↓ ↓ ↓ ↓ ↓ ↓
5 10 2 1 9 8

$\rightarrow O(N \times L) \rightarrow \text{precomputation}$

hash_map or tree_map

↓
 $O(1)$

↓
 $O(\log N)$

$x = \underline{abc} \rightarrow \text{hash}(x) \rightarrow 5$

if (hashmap.find(5) . . .)

each query

$O(x) + O(1) \quad | \quad O(x) + O(\log N)$



$$O(N \times L) + O(Q[x + \log N])$$



Part - 2

No. of strings with prefix as X

N strings

↳ $10^9 \rightarrow 30$

$10^{18} \rightarrow 60$

$$\underline{O(1 \times 1 \cdot \log N)}$$

$N \rightarrow \text{Google} \rightarrow$

$\rightarrow \underline{\underline{600}}$

$(20 \cdot 600)$

$$10^9 \times 20 \times 600$$

$$10^9 \times 20 \times \underline{600}$$

$$10^9 \times 20$$

query $\rightarrow$ ₹1

600×10^9 Rupers saved

$N_i = abcd \rightarrow \text{Beauty of Hashing}$

$X = ab$

Hash $\rightarrow abcd \rightarrow O(L)$

TC of finding a single hash ==

TC of creating a prefix hash
array

Precomputation $\rightarrow$

$$O(N \cdot L + N \cdot L \cdot \log(N \cdot L))$$

$$\underline{\underline{O(x) + O(\log(N \cdot L))}}$$

$\hookrightarrow$ one query

Part-1 (Optimised)

→ sort these strings

X

$\{a_1 \leq a_2 \leq a_3 \dots \leq a_n\}$

abcd abce abd - - - -



$X = abc$

$a[mid] > X \rightarrow \text{go left}$

$a[mid] < X \rightarrow \text{go right}$

$a[mid] == X \rightarrow \text{"wow"}$

A →

a	b	c	e			
---	---	---	---	--	--	--

↓

T T T F F F F

X →

a	b	c	d
---	---	---	---

BS on the hashes again??

True → right
false → left

$O(x) \rightarrow O(\log x)$

① Sort all strings - $O(N \log N \cdot L)$ $\nearrow \log L$

② Compute all prefix hashes - $O(N \cdot L)$

$$O(N \log N \cdot L + N \cdot L)$$

$\hookrightarrow$ query $\downarrow \log L$

$\rightarrow$ ① Compute Hash of $X \rightarrow O(|X|)$

② Do a Binary Search of N (set of strings)

$$\rightarrow O(\log N \cdot PF)$$

$$\left. \begin{array}{l} a[mid] > x \\ < x \\ == x \end{array} \right\} \rightarrow O(\log |x|)$$

$$O(\log N \cdot \log |x|)$$

T T T ↓ F
a b c d e

T T T ↓ F
a b c e f

[← | →]
mid

$A[i] == B[i]$

↓ go right

!=
↓

go left

→ custom fn →

$\log(L)$ order

Part 2 →

2

LB



3

UB



{ 2, 2, 3, 3, 4, 4, 5 }

Occurrences of 3 is there

$(UB - LB + 1) \rightarrow \text{ans}$

$\{ \overset{LB}{\text{abc}} \leq \text{abcd} \leq \overset{UB}{\text{abc}} \leq \text{abd} \dots \leq \text{zz} \}$

$(UB - LB + 1)$

↓

$(5 - 3 + 1) \Rightarrow 3$

abc

How to calculate "U"

abc

dec



$O(\log |X|)$
 $X \rightarrow$ not a prefix
 $X \rightarrow$ is prefix

$O(\log |X|)$
 $a[mid] > x$ go left
 $a[mid] < x$ go right

$\rightarrow$ go right ?? ✓

$$O(\log N \cdot \log |X|)$$



whether X
is a prefix



Overall

one query $\rightarrow$

$$O(1 \times 1) + O(\log N \cdot \log |X|)$$

Q queries =

$$O(Q [1 + \log N \cdot \log |X|])$$

$$\Rightarrow O(Q \cdot 1 + Q \log N \cdot \log |X|)$$

$O(Q \cdot X) \rightarrow$ already then
to take input
of q queries

So $\rightarrow$

$$O(Q \cdot \log N \cdot \log X)$$

$$O(\overset{\textcircled{1}}{Q} \cdot |X| \cdot \log N) \gg \gg$$

EMAIL

↳ 0.002
↓
0.001 }

$$O(N \log N \cdot L + N \cdot L) \rightarrow \text{pre computation}$$

$$\hookrightarrow O(N \cdot L) \rightarrow O(1)$$

$$O(Q[X + \log N \cdot \log X]) \text{ queries}$$

$$\hookrightarrow O(\underbrace{QX + \frac{Q \log N \log X}{X}}_X) \rightarrow O(QX) \rightarrow O(1)$$

$O(\log N) \rightarrow \text{BS}$

$O(1 \times 1) \rightarrow \text{Predicate Fn} \dots$

$O(\log N \cdot |x|)$

Problem Statement [Cont.]



- Handle updates like add a new string a_{new} or delete an old string a_{old} from the set of strings and do the following for a new string X
 - **Part 1:** Find if X exists in the set of strings
 - **Part 2:** Find number of strings with prefix equal to X



Solution

- Brute Force

✓ ○ **Time Complexity: $O(Q * N * |X|)$**

■ Reduces to **$O(Q * \log N * |X|)$** with hashing

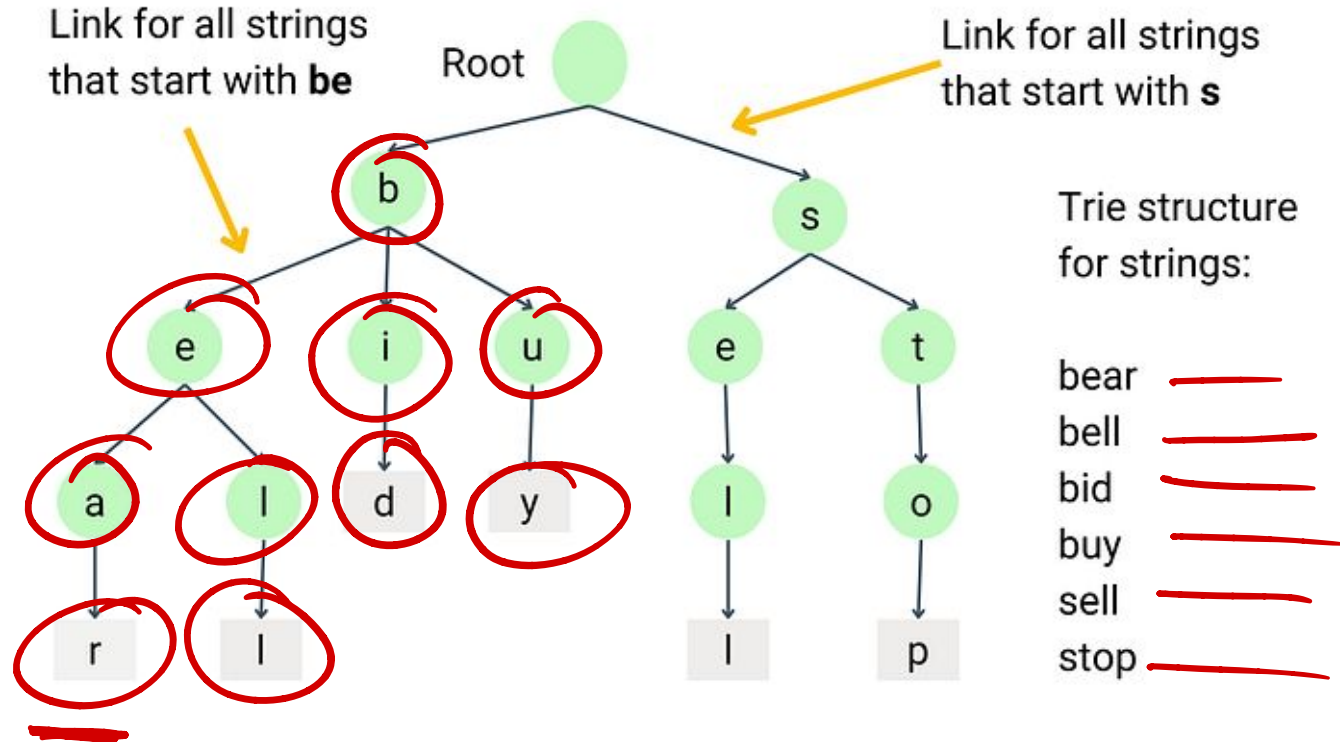
- ✓ Sorting + Binary Search

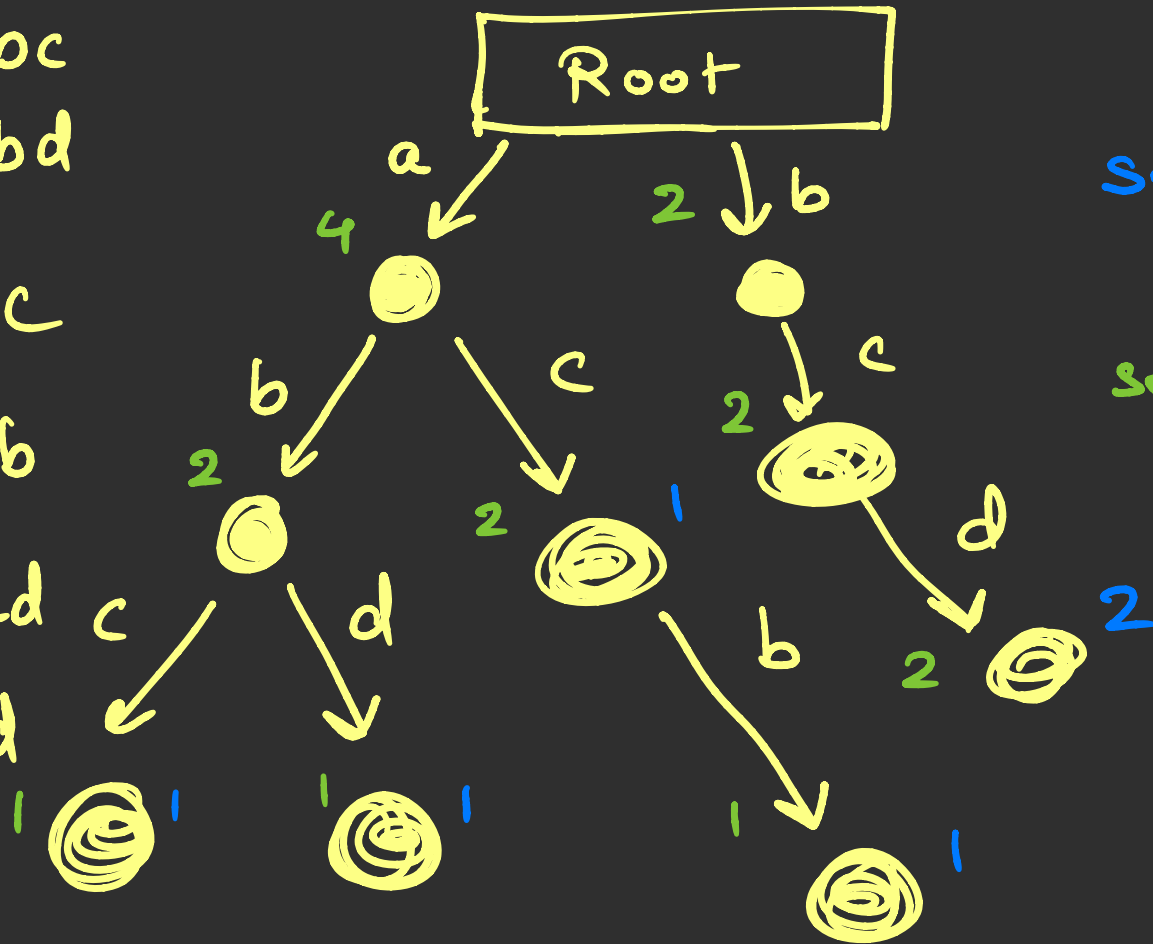
○ **Time Complexity:**

■ **$O(N \log N * L + N * L) + O(Q * |X| + Q * \log N * \log |X|)$**

- ✓ **Let's build a Trie (no, the correct spelling is not Tree)**

What is a Trie?



$$a_2 = abd$$
$$a_4 = acb$$
$$a_c = bcd$$
$$X = ab$$


Strings - ending
- here

Strings - ending
- below

$$O(N \cdot L)$$
$$O(Q \cdot x)$$



What would be a TrieNode?

```
1  class TrieNode {  →
2  public:
3      // Initialize your data structure here.
4      int strings_ending_here;  —
5      int strings_going_below;  —
6  → TrieNode *children[26];
7      TrieNode() { ←
8          strings_ending_here = 0; ←
9          strings_going_below = 0; ←
10         for (int i = 0; i < 26; i++)
11             children[i] = NULL; ←
12     }
13 };
```



What would be a Trie?

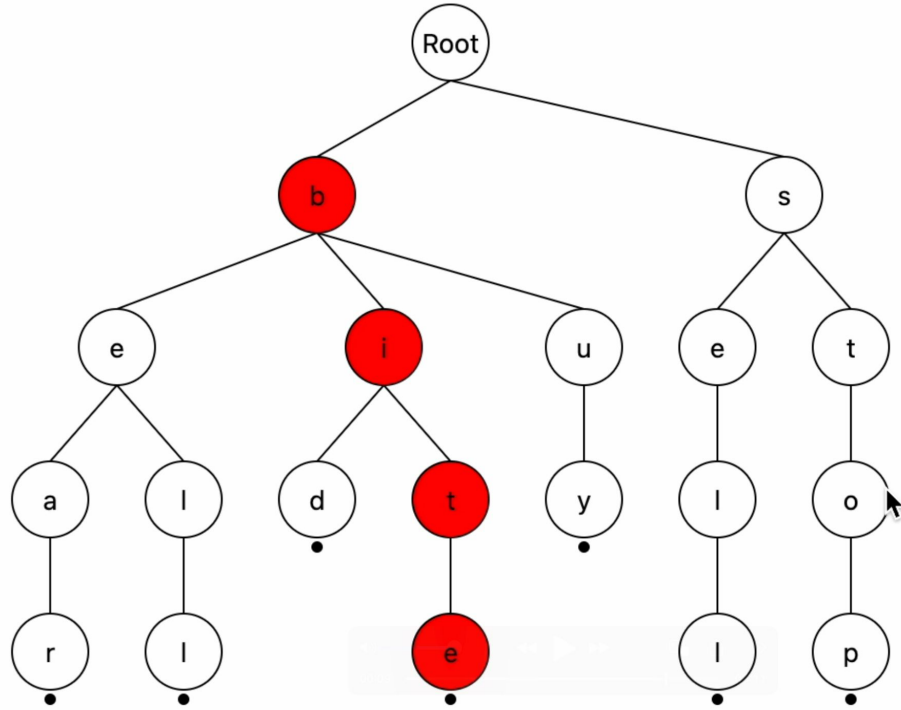
```
1  class Trie{ ←
2      public:
3      TrieNode *root; ←
4      Trie()
5      {
6          root = new TrieNode();
7      }
8      // add functions
9  }; add
    Search
```

;

Adding a new string to Trie



Adding word = “**bite**”





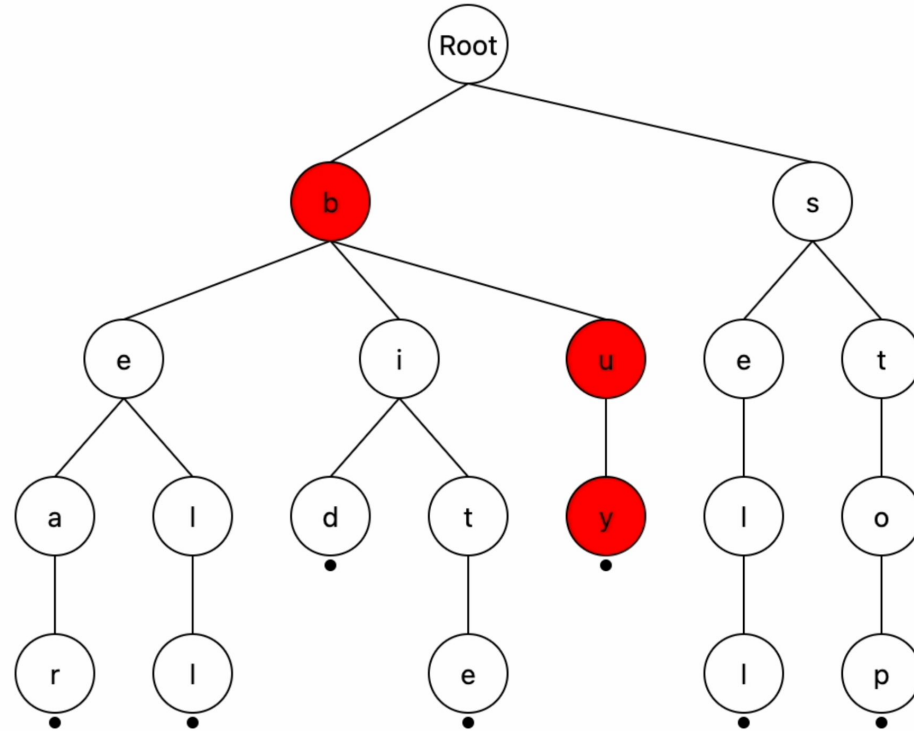
Adding a new string to Trie

```
1 // Adds a word into the trie.
2 void add(string word) {
3     TrieNode *curr = root; ←
4     for (auto ch : word)
5     {
6         if (curr->children[ch-'a'] == NULL)
7             curr->children[ch-'a'] = new TrieNode();
8         curr = curr->children[ch-'a'];
9         curr->strings_going_below++; ✓
10    }
11    curr->strings_ending_here++; ✓
12 }
```

Searching for a string in a Trie



Searching word = “**buy**”





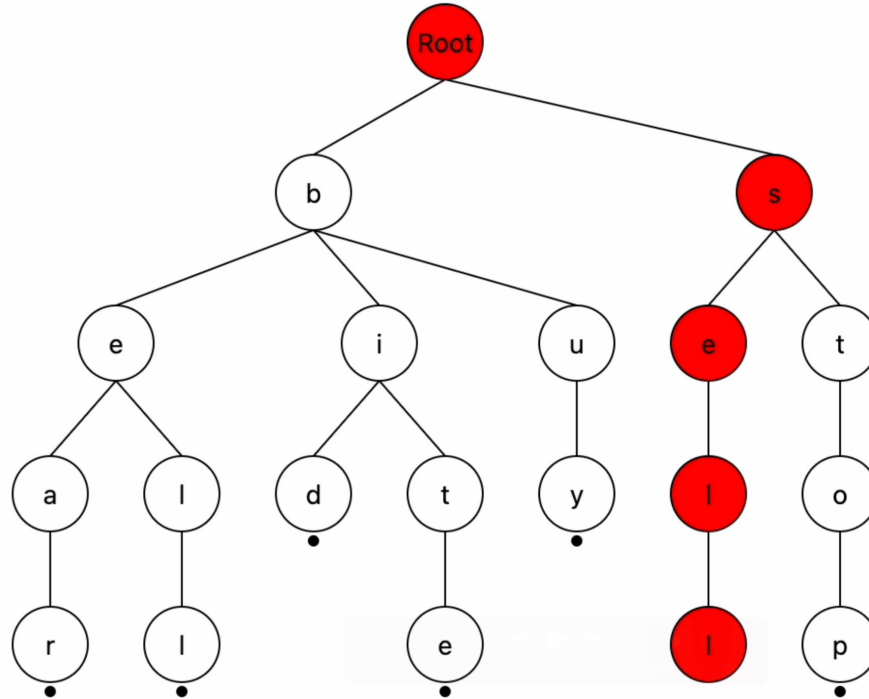
Searching for a string in a Trie

```
1 // Searches a word in the trie.
2 void bool search(string word)
3 {
4     TrieNode *curr = root;
5     for (auto ch : word) ←
6     {
7         if (curr->children[ch - 'a'] == NULL) ←
8             return false; ←
9         curr = curr->children[ch - 'a'];
10    }
11    return curr->strings_ending_here > 0;
12 }
```

Deleting an old string from a Trie



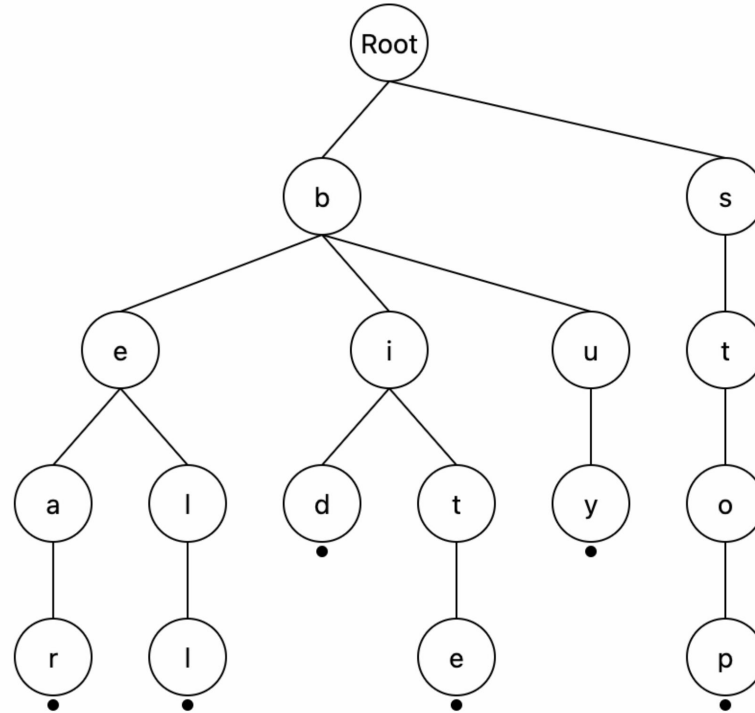
Deleting word = "sell"



Deleting an old string from a Trie



Final after deletion





Deleting an old string from a Trie

- Will lead to memory leak, or “OUT OF MEMORY”
- No garbage collection in C++
- Pointers -> Useless (somewhat)

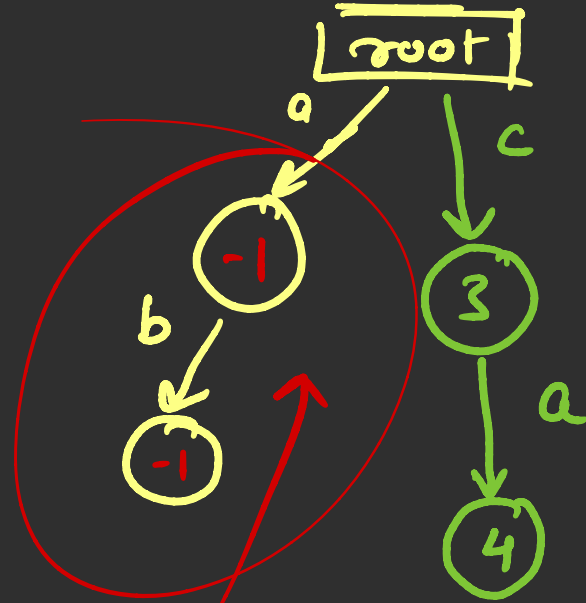
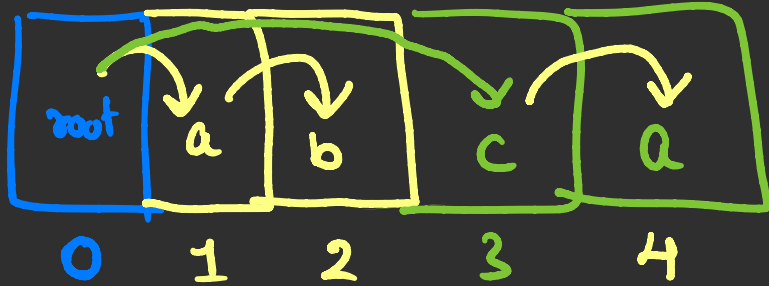


No pointer approach?

```
struct TrieNode {  
    vector < int > children(26, -1) ;  
    int strings - .. = 0 ;  
    int string - bel.. = 0 ;  
}
```

ab $\rightarrow$ ca

Trie Tree $\rightarrow$ vector





What would be a TrieNode?

```
1  const int letters = 26;
2  struct TrieNode
3  {
4      vector<int> children; ✓
5      int strings_ending_here; ✓
6      int strings_going_below; ✓
7      TrieNode()
8      {
9          children.resize(letters, -1);
10         strings_ending_here = 0;
11         strings_going_below = 0;
12     }
13 };
```



What would be a Trie?

```
1 struct Trie{ TrieNode
2     vector<Node> TrieTree; ←
3     int sizeOfTrie = 0;
4     Trie(){
5         TrieTree.push_back(Node());
6         sizeOfTrie++;
7     }
8     // add functions
9 }
```



Adding a new string to Trie

```
1 void add(string word)
2 {
3     int currNode = 0;
4     for(auto ch: word){
5         int index = ch-'a';
6         if(TrieTree[currNode].children[index] == -1){
7             TrieTree[currNode].children[index] = sizeofTrie;
8             TrieTree.push_back(Node());
9             sizeofTrie++;
10        }
11        currNode = TrieTree[currNode].children[index];
12        TrieTree[currNode].strings_going_below++;
13    }
14    TrieTree[currNode].strings_ending_here++;
15 }
```



Searching for a string in a Trie



```
1  bool search(string word)
2  {
3      int currNode = 0;
4      for(auto ch: word){
5          int index = ch-'a';
6          if(TrieTree[currNode].children[index] == -1)
7              return false;
8          currNode = TrieTree[currNode].children[index];
9      }
10     return TrieTree[currNode].strings_ending_here > 0;
11 }
```



Deleting an old string from a Trie

```
1 void delete(string word)
2 {
3     if(search(word) == false)
4         return;
5     int currNode = 0;
6     for(auto ch: word){
7         int index = ch-'a';
8         int child = TrieTree[currNode].children[index];
9         TrieTree[child].strings_going_below--;
10        if(TrieTree[child].strings_going_below == 0)
11            TrieTree[currNode].children[index] = -1;
12        currNode = child;
13    }
14    TrieTree[currNode].strings_ending_here--;
15 }
```

